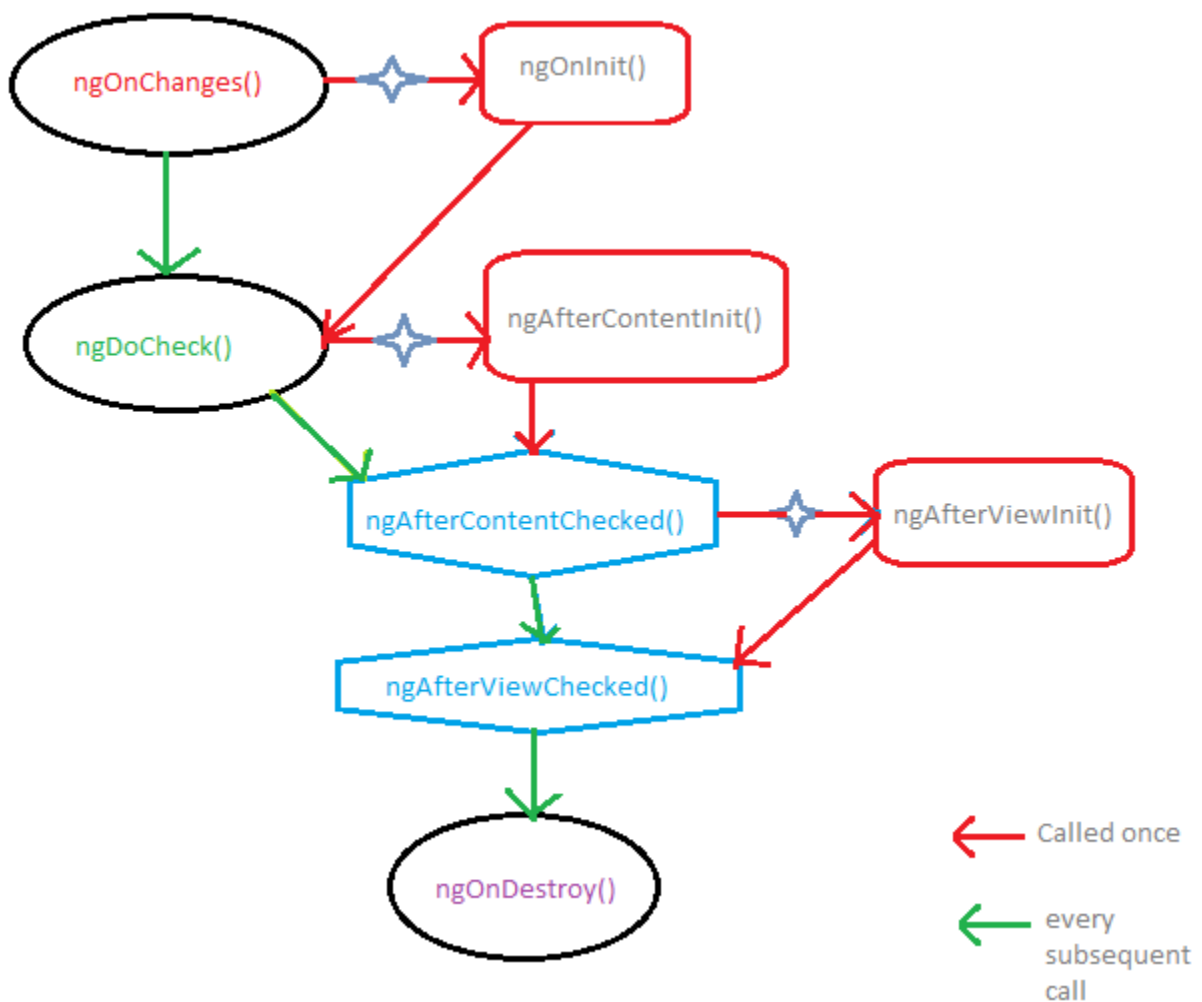


Angular Component Lifecycle methods

Angular Component Lifecycle – Concept Overview

In Angular, a **component lifecycle** represents the **sequence of events** that occur from the moment a component is **created**, **rendered**, **checked**, and finally **destroyed**.

Angular exposes these events through **lifecycle hook methods**. By implementing them, developers can execute custom logic at specific points in a component's life.



constructor

ngOnChanges

ngOnInit

ngDoCheck

ngAfterContentInit

ngAfterContentChecked

ngAfterViewInit

ngAfterViewChecked

ngOnDestroy

Why Lifecycle Hooks Are Important

Lifecycle hooks are commonly used to:

- Initialize data
- React to input (@Input) changes
- Access DOM elements safely
- Optimize performance
- Clean up subscriptions and resources

Complete List of Angular Lifecycle Hooks

Hook	Purpose
constructor()	Component instantiation
ngOnChanges()	React to @Input value changes
ngOnInit()	Component initialization
ngDoCheck()	Custom change detection
ngAfterContentInit()	Content projection initialized
ngAfterContentChecked()	Projected content checked
ngAfterViewInit()	View & child views initialized
ngAfterViewChecked()	View checked
ngOnDestroy()	Cleanup before destruction

Lifecycle Flow (High Level)

constructor



ngOnChanges



ngOnInit



ngDoCheck



ngAfterContentInit



ngAfterContentChecked



ngAfterViewInit



ngAfterViewChecked



ngOnDestroy

Practical Example – Student Profile Component

Scenario

We will build a `StudentProfileComponent` that:

- Receives data from a parent (`@Input`)
- Loads data on initialization
- Accesses the DOM safely
- Cleans up resources

Component Code (Lifecycle Hooks in Action)

`student-profile.component.ts`

```
import {  
  Component,  
  Input,  
  OnInit,  
  OnChanges,  
  SimpleChanges,  
  AfterViewInit,  
  OnDestroy,  
  ViewChild,  
  ElementRef  
} from '@angular/core';
```

```
@Component({
  selector: 'app-student-profile',
  standalone: true,
  templateUrl: './student-profile.component.html'
})
export class StudentProfileComponent
  implements OnInit, OnChanges, AfterViewInit, OnDestroy {

  @Input() studentName!: string;

  @ViewChild('title') title!: ElementRef;

  constructor() {
    console.log('Constructor: Component created');
  }

  ngOnChanges(changes: SimpleChanges): void {
    console.log('ngOnChanges:', changes);
  }

  ngOnInit(): void {
    console.log('ngOnInit: Initialization logic');

    // API calls, default values, subscriptions
  }
}
```

```
ngAfterViewInit(): void {  
  console.log('ngAfterViewInit: View initialized');  
  this.title.nativeElement.style.color = 'blue';  
}
```

```
ngOnDestroy(): void {  
  console.log('ngOnDestroy: Cleanup');  
  // unsubscribe, clear timers  
}  
}
```

student-profile.component.html

```
<h2 #title>Student Profile</h2>  
<p>Name: {{ studentName }}</p>
```

Parent Component Usage

```
<app-student-profile [studentName]="selectedStudent"></app-student-profile>
```

What Happens Step-by-Step

1. constructor

- Component instance is created
- Avoid API calls here

2. ngOnChanges

- Triggered when studentName changes
- Useful for reacting to parent data updates

3. ngOnInit

- Best place for:
 - API calls
 - Initialization logic
 - Subscriptions

4. **ngAfterViewInit**

- DOM elements (ViewChild) are accessible
- Ideal for DOM manipulation

5. **ngOnDestroy**

- Cleanup logic
- Prevents memory leaks

Common Focused Notes

- **constructor vs ngOnInit**
 - constructor: dependency injection only
 - ngOnInit: business logic
- **DOM access**
 - Never access DOM before ngAfterViewInit
- **Memory leaks**
 - Always unsubscribe in ngOnDestroy

Real-World Use Cases Mapping

Use Case	Hook
API call	ngOnInit
Detect input change	ngOnChanges
DOM manipulation	ngAfterViewInit

Use Case	Hook
Cleanup	ngOnDestroy